USER INTERFACE DESIGN EXP:3 COMMAND INTERFACES CLI, GUI, VUI

SAIPRASHANTH V 240701457

Aim

To design and implement a To-Do List Manager application using three different interfaces:

- Command Line Interface (CLI)
- Graphical User Interface (GUI)
- Voice User Interface (VUI)

and compare their usability and user satisfaction.

Objective

- To understand different types of user interfaces.
- To implement task management using CLI, GUI and VUI.
- To compare efficiency, usability and user satisfaction.
- To analyze advantages and limitations of each interface type.

Tools Used

- Python Programming Language
- Tkinter (for GUI)
- SpeechRecognition Library (for VUI)
- PyAudio (Microphone input)
- VS Code / Terminal

Theory

User Interfaces allow interaction between users and systems. There are multiple types of interfaces:

1. CLI (Command Line Interface) – Text-based interaction.
2. GUI (Graphical User Interface) – Visual interaction using buttons and windows.
3. VUI (Voice User Interface) – Interaction using speech commands.

Each interface provides different levels of usability, learning curve and efficiency.

This project demonstrates how the same application (To-Do List Manager) can be implemented using different interaction styles.

System Description

The To-Do List Manager allows users to:

- Add tasks
- Remove tasks
- Show tasks
- Clear tasks
- Navigate back to main menu

The application has a main menu screen where the user selects:

- CLI
- GUI
- VUI (Voice)
- Exit

A. CLI (Command Line Interface)

Description

The CLI version allows users to manage tasks by typing commands in a text-based terminal window.

The interface displays menu options:

1. Add Task
2. Remove Task
3. Show Tasks
4. Back to Main Menu

The user enters the corresponding number to perform operations.

How It Works

- The program stores tasks in a list.
- User inputs are read using `input()`.
- Based on the selection, tasks are added, removed, or displayed.
- Error handling prevents invalid selections.

Code

Python

```python
#!/usr/bin/env python3

# CLI To-Do List Manager

tasks = []

def show_menu():
    print("\n" + "=" * 40)
    print("      CLI TO-DO LIST MANAGER")
    print("=" * 40)
    print("1. Add Task")
    print("2. Remove Task")
    print("3. Show Tasks")
    print("4. Exit")
    print("=" * 40)

def add_task():
    task = input("Enter task: ").strip()
    if task:
        tasks.append(task)
        print(f"Task added: {task}")
    else:
        print("No task entered.")

def remove_task():
    if not tasks:
        print("No tasks to remove.")
        return

    show_tasks()
    try:
        num = int(input("Enter task number to remove: "))
        if 1 <= num <= len(tasks):
            removed = tasks.pop(num - 1)
            print(f"Task removed: {removed}")
        else:
            print("Invalid number.")
    except ValueError:
        print("Enter a valid number.")
```

```python
def show_tasks():
    if not tasks:
        print("No tasks in list.")
    else:
        print("\nCurrent Tasks:")
        for i, task in enumerate(tasks, 1):
            print(f"{i}. {task}")

def main():
    while True:
        show_menu()
        choice = input("Enter choice (1-4): ").strip()

        if choice == '1':
            add_task()
        elif choice == '2':
            remove_task()
        elif choice == '3':
            show_tasks()
        elif choice == '4':
            print("Exiting CLI...")
            break
        else:
            print("Invalid choice.")

if __name__ == "__main__":
    main()
```

B. GUI (Graphical User Interface)

Description

The GUI version uses Tkinter to create a window-based interface.

Features:

- Entry field to type task
- Add button
- Delete button
- Clear All button
- Task display area
- Back to Main button

The GUI provides a user-friendly and interactive environment.

How It Works

- Tkinter window is created using `Tk()`
- Buttons trigger functions using `command=`
- Tasks are displayed inside a listbox or text area
- Visual feedback improves usability

The GUI layout includes:

- Title: GUI Mode
- Task input field
- Scrollable task display
- Control buttons at bottom

Code

Python

```python
import tkinter as tk
from tkinter import messagebox

tasks = []

def add_task():
    task = entry.get().strip()
    if task:
        tasks.append(task)
        entry.delete(0, tk.END)
        refresh_list()
    else:
        messagebox.showwarning("Warning", "Enter a task.")

def delete_task():
    try:
        index = listbox.curselection()[0]
        tasks.pop(index)
        refresh_list()
    except IndexError:
        messagebox.showwarning("Warning", "Select a task to delete.")

def clear_all():
    if messagebox.askyesno("Clear All", "Delete all tasks?"):
        tasks.clear()
        refresh_list()

def refresh_list():
    listbox.delete(0, tk.END)
    for task in tasks:
        listbox.insert(tk.END, task)
    count_label.config(text=f"{len(tasks)} tasks")
```

```python
# Main window
root = tk.Tk()
root.title("GUI To-Do List Manager")
root.geometry("400x500")
root.configure(bg="#111111")

tk.Label(root, text="GUI Mode",
         font=("Arial", 20, "bold"),
         bg="#111111", fg="#e8d8c9").pack(pady=15)

frame = tk.Frame(root, bg="#111111")
frame.pack(pady=10)

entry = tk.Entry(frame, width=25, font=("Arial", 12))
entry.pack(side=tk.LEFT, padx=5)

tk.Button(frame, text="Add", command=add_task,
          bg="#f3701e", fg="black").pack(side=tk.LEFT)

listbox = tk.Listbox(root, width=40, height=10)
listbox.pack(pady=10)

count_label = tk.Label(root, text="0 tasks",
                       bg="#111111", fg="#e8d8c9")
count_label.pack()

btn_frame = tk.Frame(root, bg="#111111")
btn_frame.pack(pady=10)

tk.Button(btn_frame, text="Delete",
          command=delete_task).pack(side=tk.LEFT, padx=5)

tk.Button(btn_frame, text="Clear All",
          command=clear_all).pack(side=tk.LEFT, padx=5)

root.mainloop()
```

C. VUI (Voice User Interface)

Description

The VUI version allows users to manage tasks using voice commands.

Example Commands:

- "Add homework"
- "Remove homework"
- "Show tasks"
- "Back"

The interface shows:

- Voice Ready status
- Speak button
- Response display area
- Task count

How It Works

- Uses SpeechRecognition library
- Captures voice input using microphone
- Converts speech to text using Google Speech API
- Processes commands and updates task list
- Displays response on screen

Installation Required:

```
Code

pip install SpeechRecognition
pip install PyAudio
```

Code

Python

```python
import speech_recognition as sr
import pyttsx3

tasks = []

recognizer = sr.Recognizer()
engine = pyttsx3.init()

def speak(text):
    engine.say(text)
    engine.runAndWait()

def process_command(command):
    command = command.lower()

    if command.startswith("add "):
        task = command[4:]
        tasks.append(task)
        print(f"Task added: {task}")
        speak("Task added")

    elif command.startswith("remove "):
        task = command[7:]
        if task in tasks:
            tasks.remove(task)
            print(f"Task removed: {task}")
            speak("Task removed")
        else:
            print("Task not found")

    elif "show" in command:
        if not tasks:
            print("No tasks")
            speak("No tasks")
        else:
            print("Your tasks:")
            for t in tasks:
                print("-", t)
            speak(f"You have {len(tasks)} tasks")
```

```python
    elif command == "exit":
        speak("Exiting")
        return False

    else:
        print("Unknown command")

    return True

def listen():
    with sr.Microphone() as source:
        print("Listening...")
        recognizer.adjust_for_ambient_noise(source)
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print("You said:", command)
        return command
    except:
        print("Could not understand")
        return ""

def main():
    print("VUI To-Do List Manager")
    speak("Voice mode started")

    while True:
        command = listen()
        if not process_command(command):
            break

if __name__ == "__main__":
    main()
```

User Satisfaction Comparison

Based on implementation and testing:

1. Efficiency (Time to Complete Task)

- CLI: Fastest for experienced users.
- GUI: Moderate speed, visually guided.
- VUI: Depends on speech recognition accuracy.

2. Learning Curve

- CLI: High learning curve.
- GUI: Easy for beginners.
- VUI: Very easy but depends on clarity of speech.

3. Error Handling

- CLI: Limited feedback.
- GUI: Clear error messages.
- VUI: May misinterpret voice commands.

4. User Satisfaction

- CLI: Preferred by technical users.
- GUI: Most user-friendly and widely accepted.
- VUI: Innovative and hands-free, but accuracy dependent.

Advantages

- Demonstrates multiple interface types.
- Enhances understanding of HCI concepts.
- Shows practical comparison of interaction styles.
- Improves programming and UI design skills.

Limitations

- VUI depends on internet and microphone quality.
- CLI may be difficult for beginners.
- GUI requires more development effort.

Result

The To-Do List Manager was successfully implemented using CLI, GUI and VUI.

It was observed that:

- GUI provided the best overall user experience.
- CLI was fastest for experienced users.
- VUI offered hands-free interaction but required accurate speech recognition.

Conclusion

This experiment demonstrates the practical implementation of different user interfaces for the same application.

It proves that:

- Different interfaces suit different users.
- GUI provides better usability for general users.
- CLI offers speed and control.
- VUI provides modern, hands-free interaction.

Thus, interface selection plays a critical role in user satisfaction and system usability.